ASSIGNMENT 2: PYTHON FOR DATA ENGINEERING - The Data Pipeline Builder

Topics Covered: Python Data Structures, Functions, File Handling, Pandas, Error Handling, Performance Tuning

Duration: 8-9 hours | **Questions**: 35

Problem Statement

Build a **Production-Grade ETL Framework** called "PyFlow" that handles multiple file formats, validates data, transforms it, and loads to databases. Must be configurable, logged, and error-resilient.

Datasets

NYC Taxi Trip Data + COVID-19 Data

- > **Link 1**: https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page (Yellow Taxi Trip Records)

- > **Link 2**: https://github.com/nytimes/covid-19-data (COVID-19 time series)

- > **Link 3**: https://github.com/datasets/gdp/blob/main/data/gdp.csv (GDP data in CSV)

- **> Size**: Multiple CSV files (100MB - 500MB), JSON, Parquet formats

- **> Bonus:** https://github.com/jpatokal/openflights (Airport/airline data in CSV) [or just search openflights airline data in csv]-you will find the right repo or data

Sample files to work with from links:

- CSV: yellow_tripdata_2024-01.csv (500MB, 3M+ records)

- JSON: Nested COVID-19 data with variable schemas

- Parquet: Pre-converted taxi data

- Excel: Economic indicators (multi-sheet)

Assignment Questions (35 Total)

Part A: Python Fundamentals (7 questions)

1. Create a config parser that reads YAML/JSON config files for ETL settings (file paths, db credentials, chunk size)

2. Implement custom exception hierarchy: DataSourceError, ValidationError, TransformationError, LoadError

3. Write a decorator `@timing_decorator` to measure and log execution time of any function

4. Implement a generator function to read 500MB CSV file line-by-line without loading entire file in memory

5. Use list/dict comprehensions to extract email domains from customer data and count frequency

6. Create a context manager for database connections with automatic commit/rollback

7. Implement type hints for all functions and validate using mypy

Part B: Data Structures & Collections (5 questions)

8. Use `Counter` to find top 10 most frequent pickup locations from taxi data

9. Use `defaultdict` to group taxi trips by hour of day and calculate average fare

10. Use `deque` to implement a rolling data buffer with max 1000 records

11. Compare performance: list vs set for membership checking with 1M location IDs (benchmark and plot)

12. Design a custom class `TripRecord` with `__eq__`, `__hash__`, `__repr__` for efficient deduplication

Part C: File Handling Mastery (8 questions)

13. Read 500MB CSV in chunks of 100K rows without memory overflow

14. Auto-detect file encoding (UTF-8, Latin-1, CP1252) and read accordingly

15. Handle corrupted CSV: skip bad rows, log them to separate error file with line numbers

16. Parse nested JSON (3+ levels deep) and flatten to tabular format

17. Read Excel file with multiple sheets, merge them into single DataFrame

18. Handle compressed files (gzip, zip) - read and process without extracting to disk

19. Read Parquet file and convert to CSV preserving all data types

20. Implement file watcher: monitor directory and trigger processing on new file arrival

Part D: Pandas Deep Dive (10 questions)

21. Load 3M records CSV optimizing memory: specify dtypes, use category for low-cardinality columns

22. Handle missing values intelligently:

- Numeric: mean/median based on distribution
- Categorical: mode or 'Unknown'
- Time: forward-fill for time series

23. Detect and handle outliers using IQR method for fare amounts

24. Remove duplicates based on business keys (trip_start_time + pickup_location + dropoff_location)

25. Merge 3 DataFrames (trips, weather, holidays) using different join types, validate merge quality

26. Group by multiple columns (date, hour, borough) and aggregate with custom functions

27. Handle date/time: parse multiple formats, extract features (hour, day_of_week, is_weekend, is_holiday)

28. Implement window functions: 7-day rolling average of trip count by location

29. Use `.query()` method for complex filtering: trips during rush hours on weekdays with fare > $20

30. Export to multiple formats: CSV, JSON, Parquet, Excel with appropriate compression

Part E: Building PyFlow Framework (5 questions)

31. **Extract Module**: Create `extractor.py` with classes for CSVExtractor, JSONExtractor, ParquetExtractor (inheritance/polymorphism)

32. **Transform Module**: Create `transformer.py` with data cleaning, validation, and enrichment functions

33. **Load Module**: Create `loader.py` to load data to PostgreSQL/MySQL with batch inserts (1000 rows/batch)

34. **Config Management** : Use YAML config file to control entire ETL flow (which files, transformations, destination)

35. **Main Orchestrator**: Create `main.py` that ties everything together with proper error handling and logging

####Part F: Error Handling & Testing (5 questions)

36. Implement comprehensive logging with rotating file handler (INFO, DEBUG, ERROR levels)

37. Add retry logic with exponential backoff for database connections

38. Write 10+ unit tests using pytest for critical functions (>80% code coverage)

39. Use `pdb` to debug a provided buggy transformation function

40. Implement data validation: check email format, phone format, date ranges before loading

Bonus Challenges (3 optional)

41. Build CLI tool: `python pyflow.py --config config.yaml --mode full/incremental`

42. Implement data profiling: generate HTML report with nulls, uniques, distributions, correlations

43. Add progress bars using `tqdm` for long-running operations

Deliverables

- Python package structure:

```
pyflow/
├── pyflow/
│   ├── __init__.py
│   ├── extractors.py
│   ├── transformers.py
│   ├── loaders.py
│   ├── validators.py
│   └── utils.py
├── tests/
├── config/
├── requirements.txt
└── README.me```
```

- Unit test suite

- README with setup and usage

- Demo video showing full ETL run